

Bars - axial loads only (no shear or bending moment)
 Beam - shear and moment only (no axial)
 Frames - shear, bending, and axial loading.

FRAME ELEMENTS

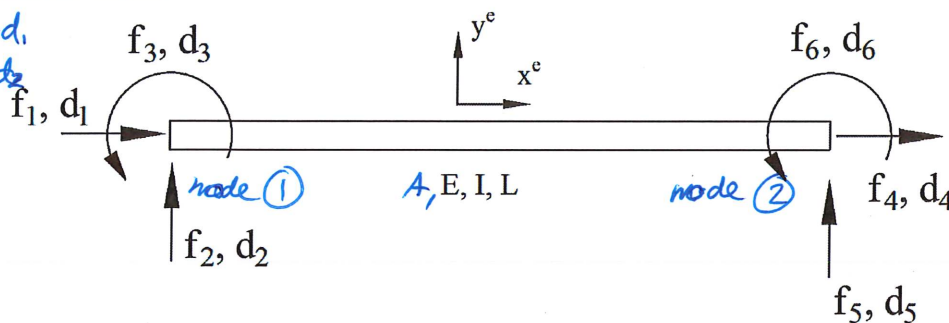
We now know how to model bar elements that carry only axial loads, and how to model beam elements that only carry shear loads and moments. For a general element, we want to be able to consider all possible loads simultaneously. We will now define FRAME elements, which are able to support axial and shear forces, as well as bending moments.

Does that mean that we need to do all those integrals and shape functions again?? Thankfully, NO!

The stiffness matrix for a FRAME element can be obtained from the Principle of Linear Superposition.

A 6 DOF FRAME Element:

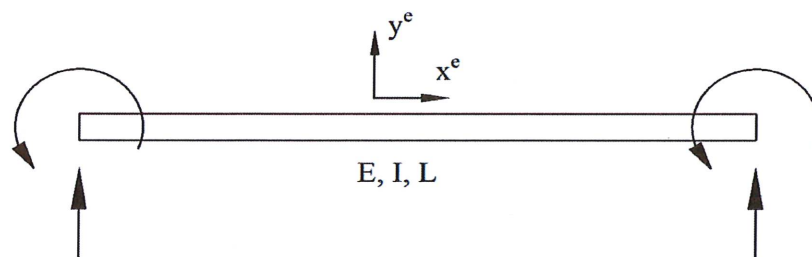
x^e at node ① = d_1
 y^e at node ① = d_2
 z^e at node ① = d_3



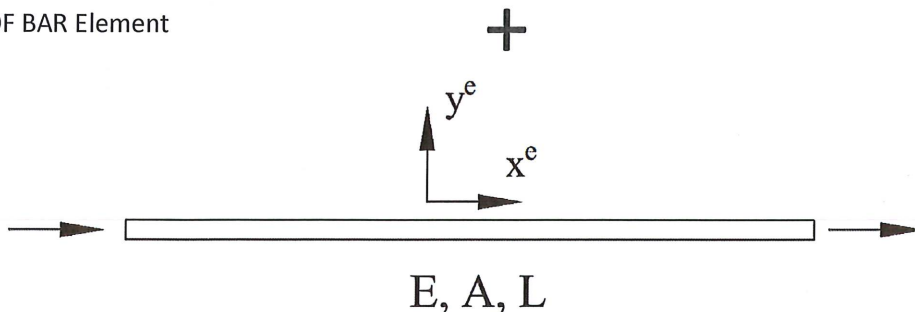
Assumes small deflections.
 Assumes linear elastic behaviour.

x^e at ② $\Rightarrow d_4$
 y^e at ② $\Rightarrow d_5$
 z^e at ② $\Rightarrow d_6$

is simply the addition of a 4 DOF BEAM Element: =



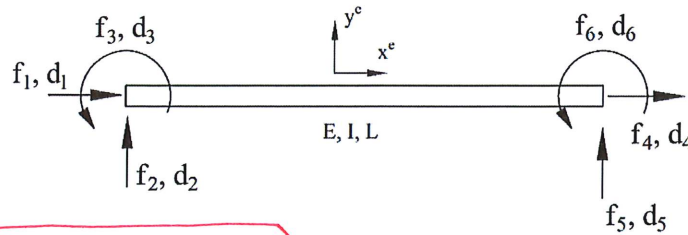
and a 2 DOF BAR Element



Assumption: No coupling between axial terms (d_1, d_4 , and f_1, f_4) and flexural (bending) terms (d_2, d_3, d_5, d_6 , and f_2, f_3, f_5, f_6). Each set of terms can be calculated independently.

What does a FRAME element stiffness matrix look like?

element stiffness matrix in element coordinates.



$$f_{\{6 \times 1\}}^e = K_{\{6 \times 6\}}^e d_{\{6 \times 1\}}^e \rightarrow \text{in element coordinate system}$$

Frame Element Stiffness Matrix in Local Co-ordinates:

$$K_{\{6 \times 6\}}^e = \frac{EI}{L^3} \begin{bmatrix} \beta & 0 & 0 & -\beta & 0 & 0 \\ 0 & 12 & 6L & 0 & -12 & 6L \\ 0 & 6L & 4L^2 & 0 & -6L & 2L^2 \\ -\beta & 0 & 0 & \beta & 0 & 0 \\ 0 & -12 & -6L & 0 & 12 & -6L \\ 0 & 6L & 2L^2 & 0 & -6L & 4L^2 \end{bmatrix} \quad \text{where: } \beta = \frac{AL^2}{I}$$

These zeroes exist because there is no coupling between axial and transverse/moment terms.

Again, the stiffness matrix above is in local (element) co-ordinates. However, it can be transformed into global co-ordinates in a very similar way to a bar element. The stiffness matrix for a FRAME element, in global co-ordinates is calculated using the following (hopefully familiar looking) equation:

$$\bar{K}_{\{6 \times 6\}}^e = \Lambda_{\{6 \times 6\}}^T K_{\{6 \times 6\}}^e \Lambda_{\{6 \times 6\}}^e$$

stiffness equation transformed in global coordinates.

But now the transformation matrix looks a little different:

Frame Element Transformation Matrix

$$\Lambda_{\{6 \times 6\}}^e = \begin{bmatrix} c & s & 0 & 0 & 0 & 0 \\ -s & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s & 0 \\ 0 & 0 & 0 & -s & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \lambda_{\{3 \times 3\}} & 0_{\{3 \times 3\}} \\ 0_{\{3 \times 3\}} & \lambda_{\{3 \times 3\}} \end{bmatrix} \quad \text{where: } \lambda_{\{3 \times 3\}} = \begin{bmatrix} c & s & 0 \\ -s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$c = \cos \alpha \\ s = \sin \alpha$$

In a 2D plane, any 2D transformation will still have z^e upwards out of the page.

↑ four 3x3 submatrices form the overall 6x6 np.block.

⇒ In a 2D plane, z is always z . No transformation needed, hence 1-0.

All the other relationships remain the same as before (but now the vectors/matrices are larger):

$$\begin{aligned}
 f_{\{6 \times 1\}}^e &= K_{\{6 \times 6\}}^e d_{\{6 \times 1\}}^e \\
 F_{\{6 \times 1\}}^e &= \hat{K}_{\{6 \times 6\}}^e D_{\{6 \times 1\}}^e \\
 d_{\{6 \times 1\}}^e &= A_{\{6 \times 6\}}^e D_{\{6 \times 1\}}^e \\
 F_{\{6 \times 1\}}^e &= A_{\{6 \times 6\}}^{eT} f_{\{6 \times 1\}}^e
 \end{aligned}$$

All the matrix equations are exactly the same as what we used for bar elements. The size of the matrices/vectors has now changed, but the way that they multiply together remains unchanged.

Furthermore, assembly of a structure using FRAME elements is also just like before:

$$\begin{aligned}
 D_{\{6 \times 1\}}^e &= (A^e)^T_{\{6 \times n\}} q_{\{n \times 1\}} \\
 Q_{\{n \times 1\}} &= \sum_{e=1}^{n_e} A_{\{n \times 6\}}^e F_{\{6 \times 1\}}^e \\
 K_{G\{n \times n\}} &= \sum_{e=1}^{n_e} A_{\{n \times 6\}}^e \hat{K}_{\{6 \times 6\}}^e (A^e)^T_{\{6 \times n\}} \\
 n_e &- \text{total number of elements} \\
 n &- \text{total number of overall (structural) DOF}
 \end{aligned}$$

For a FRAME element

columns = # element DOFs in global co-ordinates
(6 DOF for a FRAME element)

rows = # global DOFs
in the overall structure
(the # of q's)

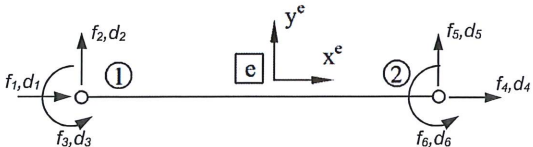
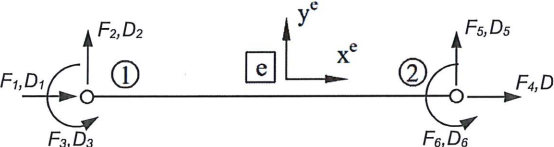
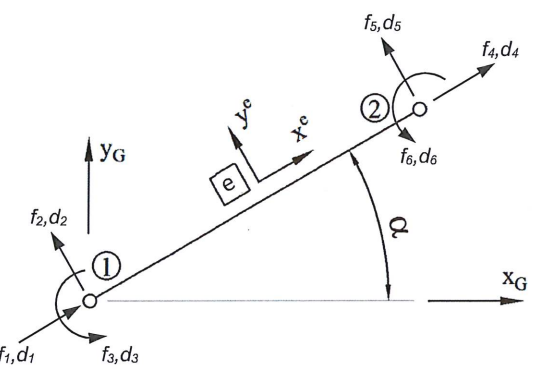
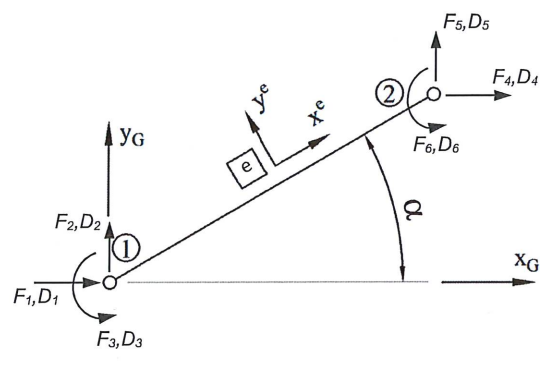
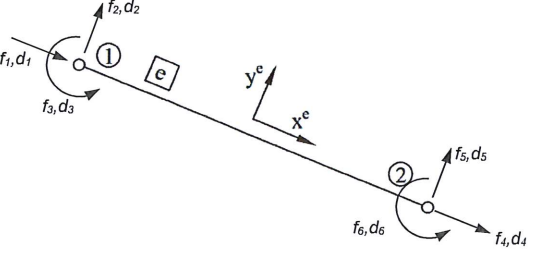
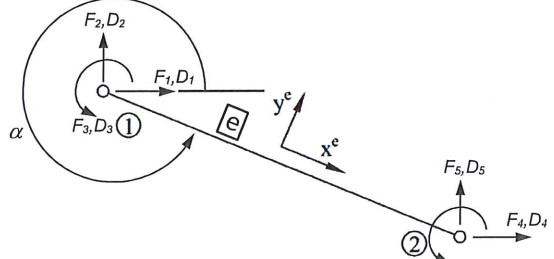
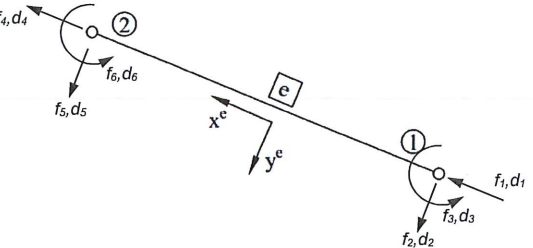
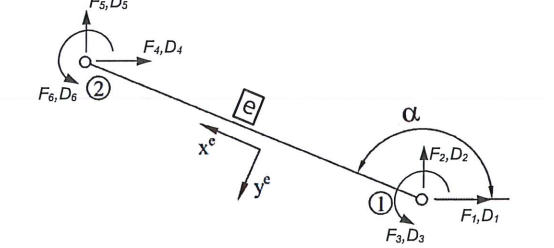
Element Assembly Matrix

And just like before:

$$Q_{\{n \times 1\}} = K_{G\{n \times n\}} q_{\{n \times 1\}}$$

Note: you can make a FRAME element behave like a BEAM element in an assembly, simply by constraining the axial degrees of freedom (d_1 and d_4), enforcing the element to be axially rigid and preventing any axial deformation. Therefore, the FRAME element essentially supersedes the BEAM element and we will deal with FRAME elements from here.

Local vs Global Element Degrees of Freedom for a FRAME element

Element DOF in local (element) co-ordinates	Element DOF in global co-ordinates
Local x^e at node 1 $\rightarrow d_1$ Local y^e at node 1 $\rightarrow d_2$ Local z^e at node 1 $\rightarrow d_3$ Local x^e at node 2 $\rightarrow d_4$ Local y^e at node 2 $\rightarrow d_5$ Local z^e at node 2 $\rightarrow d_6$	Global X^G at node 1 $\rightarrow D_1$ Global Y^G at node 1 $\rightarrow D_2$ Global Z^G at node 1 $\rightarrow D_3$ Global X^G at node 2 $\rightarrow D_4$ Global Y^G at node 2 $\rightarrow D_5$ Global Z^G at node 2 $\rightarrow D_6$
	
	
	
	

$\alpha^e = 0^\circ$

$\alpha = 30^\circ$

$\alpha = 340^\circ$
or $\alpha = -20^\circ$

$\alpha = 160^\circ$

Again, you can choose any element end as a start node #1, which will define $f_1, f_2, f_3, d_1, d_2, d_3$
 The angle of rotation, α , will depend on the choice of start node #1 and end node #2.

Summary of the Steps in the Finite Element Solution Method

- (1) Determine the allowable global degrees of freedom (the qs) and the corresponding global forcing terms (the Qs) based upon the number of nodes in the overall structure and the type of support points. Label and number these on a diagram of the overall structure using a systematic approach.
- (2) Draw free-body diagrams of each element, labelling the element displacements and forcing terms in global co-ordinates (the D^e and F^e components) as well as the angle of orientation, α^e .
- (3) Create the element stiffness matrix in local co-ordinates, K^e , from E , A , I , and L .
- (4) Using the element orientation angle α^e , create the element transformation matrix, Λ^e , and use that to generate the bar element stiffness matrix in global co-ordinates, $\hat{K}^e = \Lambda^{eT} K^e \Lambda^e$
- (5) Write out/create the assembly matrix, A^e , for each element by hand (by inspection).
- (6) Create the individual element contributions to the global stiffness matrix:

$$K_G^e_{\{n \times n\}} = A^e_{\{n \times 6\}} \hat{K}^e_{\{6 \times 6\}} (A^e)^T_{\{6 \times n\}}$$

- (7) Sum up these individual element contributions to get the overall global stiffness matrix:

$$K_G_{\{n \times n\}} = \sum_{e=1}^{n_e} A^e_{\{n \times 6\}} \hat{K}^e_{\{6 \times 6\}} (A^e)^T_{\{6 \times n\}}$$

n_e – total number of elements
 n – total number of overall (structural) DOF

- (8) Solve the overall structural equation $q = (K_G)^{-1} Q$ to get the solved displacement vector, q .
- (9) Calculate element nodal forces in global co-ordinates for any given element so you can pick out and solve reaction loads.

$$F^e = \hat{K}^e D^e = \hat{K}^e (A^e)^T q$$

- (10) Calculate element nodal displacements and forces in local co-ordinates to enable stress and strain calculations:

$$d^e = \Lambda^e D^e = \Lambda^e (A^e)^T q$$

$$f^e = K^e d^e$$

Problem 4 – A simple FRAME element structure

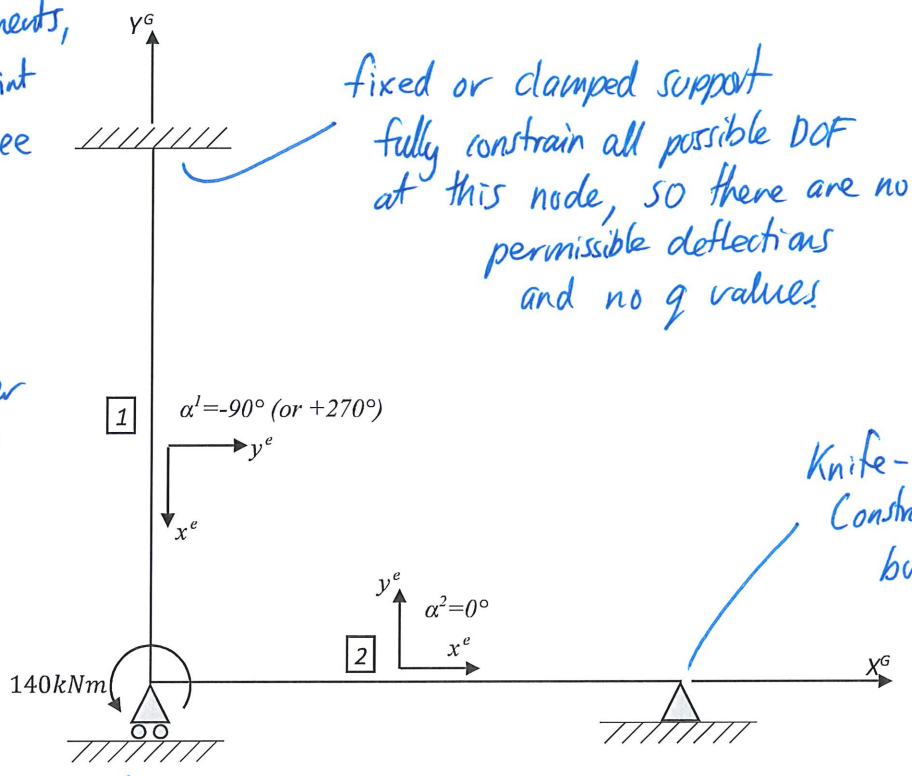
A simple structure consists of two FRAME elements. The top support is a fixed/clamped support, the bottom left support is a knife-edge support on a roller (able to provide a vertical reaction force, but not any horizontal force or moment) and the support at the bottom right is a pin-support that prevents translation but not rotation.

The only external load applied to the structure is a 140kNm concentrated moment acting at the bottom left corner. Find the structural deflections, the reaction forces at the three supports and the element internal forces.

Assume $L_1 = L_2 = 10\text{m}$, and both members have $I_1 = I_2 = 5 \times 10^{-4} \text{ m}^4$, $A_1 = A_2 = 1 \times 10^{-5} \text{ m}^2$ and the same elastic modulus of $E = 200 \text{ GPa}$.

For FRAME elements, each nodal point begins with three possible DOF. (the q values) (X_G, Y_G, Z_G)

We then consider the supports to constrain and remove DOF as appropriate.

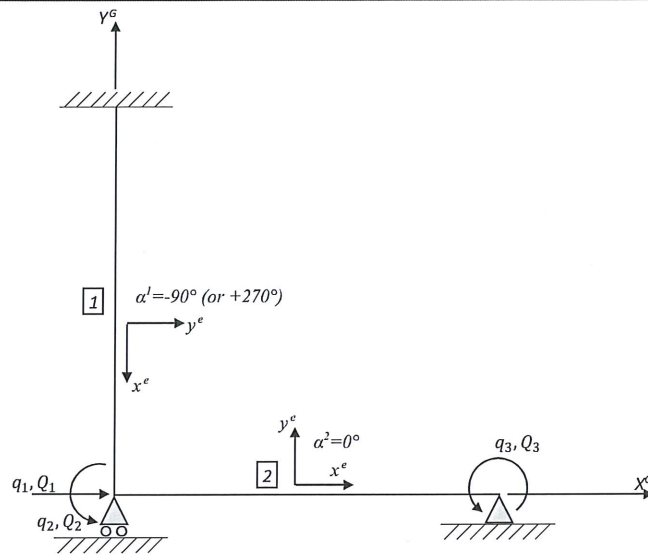


Vertical deflection (Y_G) direction is fully constrained, so no q value vertically. However, possible X_G translation and Z_G rotation so there are 2 q values here.

Solution

Determine (and label) the allowable overall structural degrees of freedom:

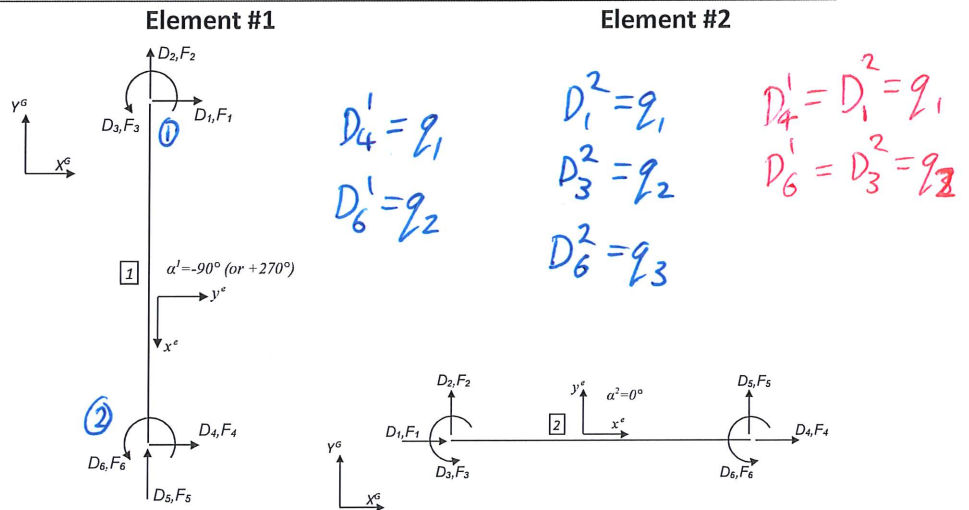
- Global structure
- Identification of DOFs, q
- Identifying external loads, Q



Element Freebody Diagrams:

Break the structure into elements

Identify element DOFs, D and angle of rotation, α



Transformation matrix

$$\Lambda_{\{6 \times 6\}}^e = \begin{bmatrix} c & s & 0 & 0 & 0 & 0 \\ -s & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s & 0 \\ 0 & 0 & 0 & -s & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} \alpha &= -90^\circ \text{ or } +270^\circ \\ c &= \cos(-90^\circ) = 0 \\ s &= \sin(-90^\circ) = -1 \end{aligned}$$

$$\Lambda_{\{6 \times 6\}}^1 = \begin{bmatrix} 0 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} \alpha &= 0^\circ \\ c &= \cos \alpha = \cos(0^\circ) = 1 \\ s &= \sin \alpha = \sin(0^\circ) = 0 \end{aligned}$$

$$\Lambda_{\{6 \times 6\}}^2 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Element stiffness matrix in local (element) co-ordinates	$K^1_{\{6 \times 6\}} = K^2_{\{6 \times 6\}} = 1 \times 10^5$		$\begin{bmatrix} 2 & 0 & 0 & -2 & 0 & 0 \\ 0 & 12 & 60 & 0 & -12 & 60 \\ 0 & 60 & 400 & 0 & -60 & 200 \\ -2 & 0 & 0 & 2 & 0 & 0 \\ 0 & -12 & -60 & 0 & 12 & -60 \\ 0 & 60 & 200 & 0 & -60 & 400 \end{bmatrix}$	
Element stiffness matrix in global co-ordinates	$\hat{K}^1 = 1 \times 10^5$	$\begin{bmatrix} 12 & 0 & 60 & -12 & 0 & 60 \\ 0 & 2 & 0 & 0 & -2 & 0 \\ 60 & 0 & 400 & -60 & 0 & 200 \\ -12 & 0 & -60 & 12 & 0 & -60 \\ 0 & -2 & 0 & 0 & 2 & 0 \\ 60 & 0 & 200 & -60 & 0 & 400 \end{bmatrix}$	$\hat{K}^2 = 1 \times 10^5$	$\begin{bmatrix} 2 & 0 & 0 & -2 & 0 & 0 \\ 0 & 12 & 60 & 0 & -12 & 60 \\ 0 & 60 & 400 & 0 & -60 & 200 \\ -2 & 0 & 0 & 2 & 0 & 0 \\ 0 & -12 & -60 & 0 & 12 & -60 \\ 0 & 60 & 200 & 0 & -60 & 400 \end{bmatrix}$
Element Assembly Matrices	$A^1 = q_1 \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$		$A^2 = q_2 \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$	
which means,		which means,		
$D_4^1 = q_1 \text{ and } D_6^1 = q_2$		$D_1^2 = q_1, D_3^2 = q_2 \text{ and } D_6^2 = q_3$		

Create the Global Stiffness Matrix (K_G) by assembling element global stiffness matrices:

$$K_G = \sum_{e=1}^{n_e} A^e \hat{K}^e A^{eT} = \underbrace{A^1 \hat{K}^1 A^{1T}}_{K_G^1} + \underbrace{A^2 \hat{K}^2 A^{2T}}_{K_G^2}$$

$$K_G = 1 \times 10^5 \begin{bmatrix} 14 & -60 & 0 \\ -60 & 800 & 200 \\ 0 & 200 & 400 \end{bmatrix}$$

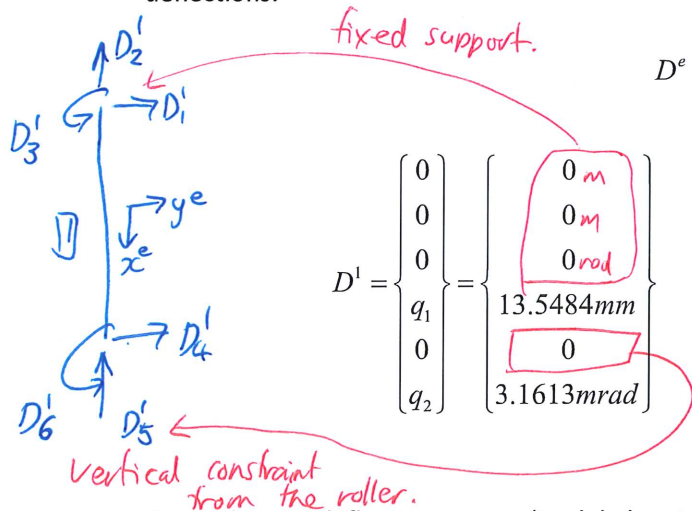
Define the global forcing vector, Q, from the information given in the question:

$$Q = \begin{Bmatrix} 0 \\ 140,000 \\ 0 \end{Bmatrix} \begin{matrix} N \\ N/m \\ N/m \end{matrix}$$

Now the structure can be solved for displacements:

$$\begin{Bmatrix} q_1 \\ q_2 \\ q_3 \end{Bmatrix} = K_G^{-1} Q = \begin{Bmatrix} 0.0135 \text{ m} \\ 0.0032 \text{ rad} \\ -0.0016 \text{ rad} \end{Bmatrix} = \begin{Bmatrix} 13.5484 \text{ mm} \\ 3.1613 \text{ mrad} \\ -1.5806 \text{ mrad} \end{Bmatrix}$$

Now that the system is solved, we can extract the relevant 'pieces' to determine the element deflections:

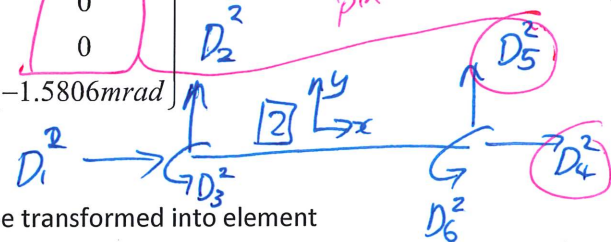


$$D^e = (A^e)^T q$$

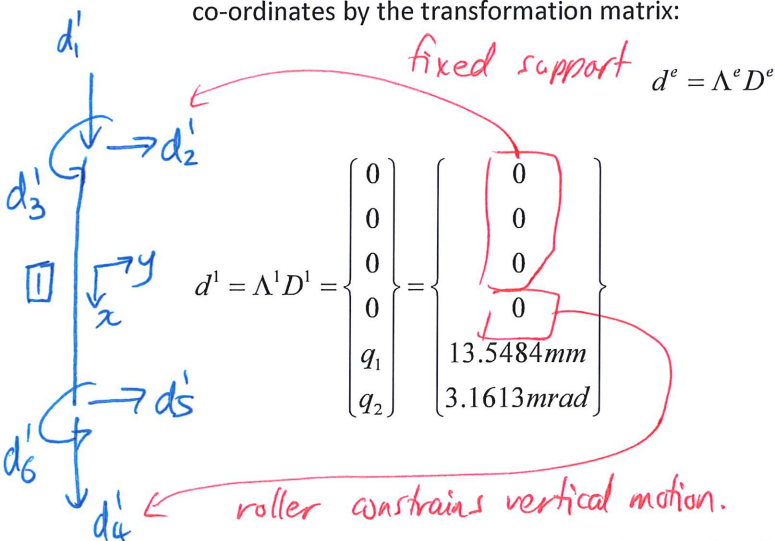
Assembly matrix extracts relevant parts of q and assigns them to the appropriate location within D vector.

$$D^1 = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ q_1 \\ 0 \\ q_2 \end{Bmatrix} = \begin{Bmatrix} 0 \text{ m} \\ 0 \text{ m} \\ 0 \text{ rad} \\ 13.5484 \text{ mm} \\ 0 \\ 3.1613 \text{ mrad} \end{Bmatrix}$$

$$D^2 = \begin{Bmatrix} q_1 \\ 0 \\ q_2 \\ 0 \\ 0 \\ q_3 \end{Bmatrix} = \begin{Bmatrix} 13.5484 \text{ mm} \\ 0 \\ 3.1613 \text{ mrad} \\ 0 \\ 0 \\ -1.5806 \text{ mrad} \end{Bmatrix}$$



These element deflection vectors (in global co-ordinates) can now be transformed into element co-ordinates by the transformation matrix:



$$d^1 = \Lambda^1 D^1 = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ q_1 \\ q_2 \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 13.5484 \text{ mm} \\ 3.1613 \text{ mrad} \end{Bmatrix}$$

$$d^2 = \Lambda^2 D^2 = \begin{Bmatrix} q_1 \\ 0 \\ q_2 \\ 0 \\ 0 \\ q_3 \end{Bmatrix} = \begin{Bmatrix} 13.5484 \text{ mm} \\ 0 \\ 3.1613 \text{ mrad} \\ 0 \\ 0 \\ -1.5806 \text{ mrad} \end{Bmatrix}$$

The element forcing vectors can now be calculated by $f^e = K^e d^e$

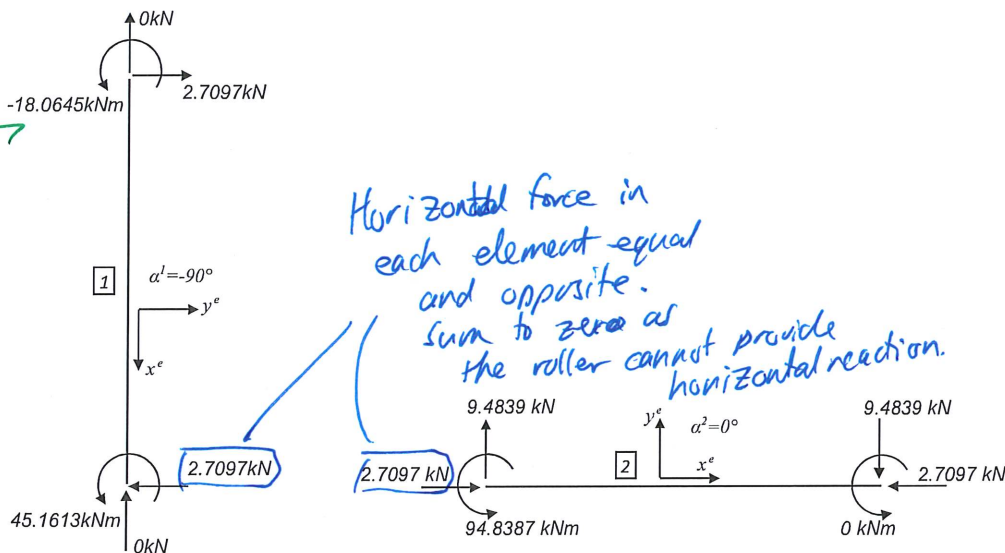
$$f^1 = K^1 d^1 = \begin{Bmatrix} 0 \text{ kN} \\ 2.7097 \text{ kN} \\ -18.0645 \text{ kNm} \\ 0 \text{ kN} \\ -2.7097 \text{ kN} \\ 45.1613 \text{ kNm} \end{Bmatrix}$$

$$f^2 = K^2 d^2 = \begin{Bmatrix} 2.7097 \text{ kN} \\ 9.4839 \text{ kN} \\ 94.8387 \text{ kNm} \\ -2.7097 \text{ kN} \\ -9.4839 \text{ kN} \\ 0 \text{ kNm} \end{Bmatrix}$$

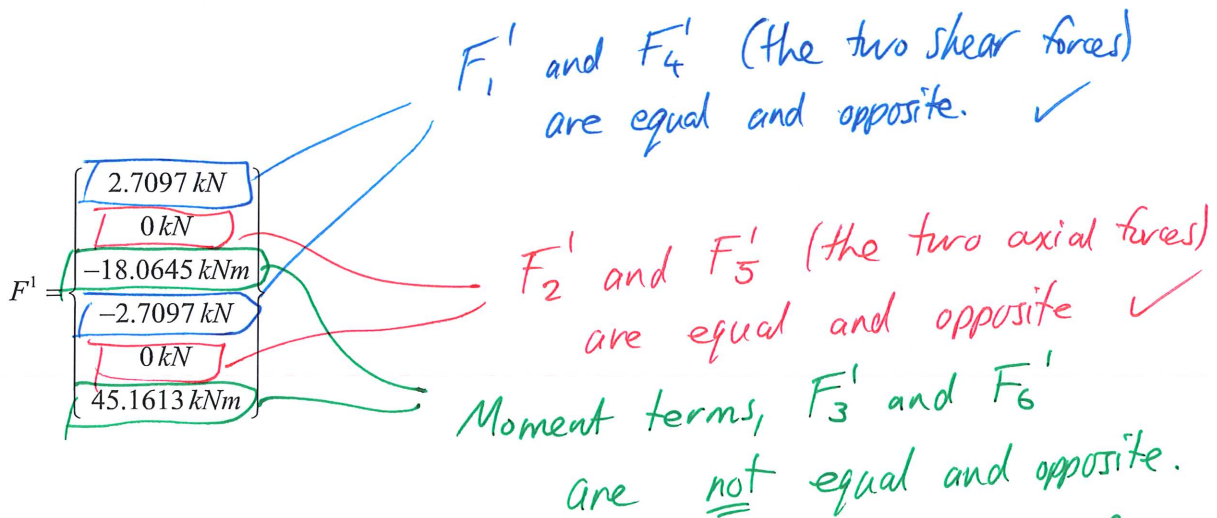
Reaction loads: When adding element forcing terms to calculate reaction loads, it is best to use element force vectors global co-ordinates (F^e), which can be calculated two ways: $F^e = \bar{K}^e D^e$ or $F^e = (\Lambda^e)^T f^e$:

$$F^1 = (\Lambda^1)^T f^1 = (\Lambda^1)^T (K^1 d^1) = \begin{Bmatrix} 2.7097 \text{ kN} \\ 0 \text{ kN} \\ -18.0645 \text{ kNm} \\ -2.7097 \text{ kN} \\ 0 \text{ kN} \\ 45.1613 \text{ kNm} \end{Bmatrix} \quad F^2 = (\Lambda^2)^T f^2 = (\Lambda^2)^T (K^2 d^2) = \begin{Bmatrix} 2.7097 \text{ kN} \\ 9.4839 \text{ kN} \\ 94.8387 \text{ kNm} \\ -2.7097 \text{ kN} \\ -9.4839 \text{ kN} \\ 0 \text{ kNm} \end{Bmatrix}$$

Using the original element free-body diagrams, we can plot these forces onto an element diagram:



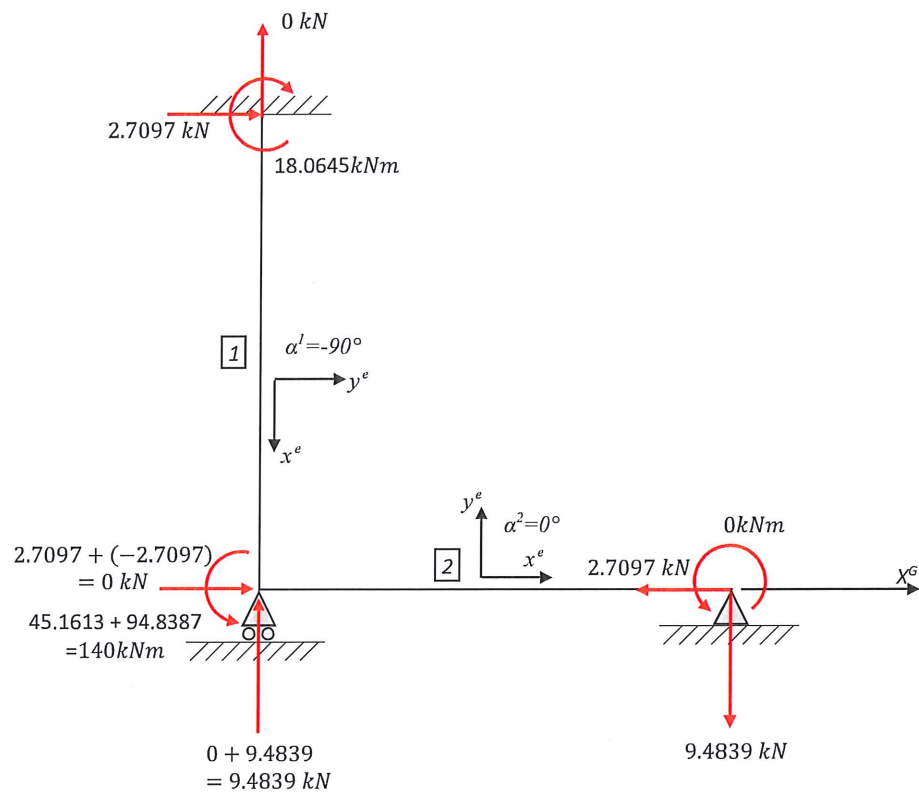
pin support at the right hand node, incapable of moment reaction. As element 2 is the only member connected to this pin, it has no moment.



The two shear forces create a force couple, and contribute a moment.

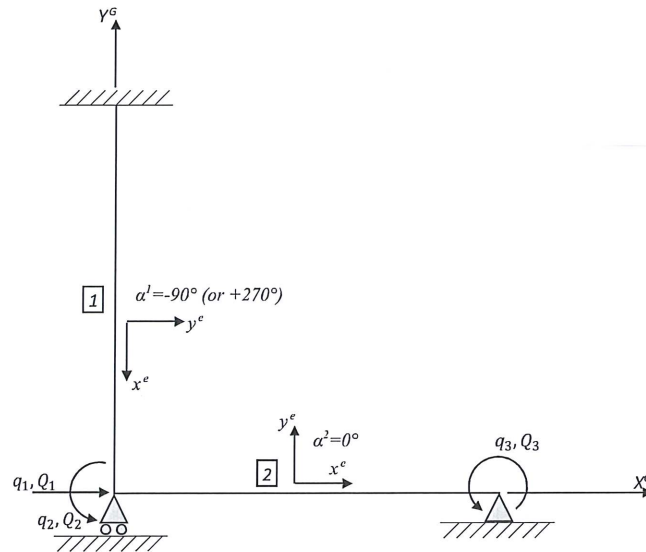
$$\sum M_{\text{node } 2}: (-18.0645) + (45.1613) - (2.7097)(10) = 0.0178 \text{ kNm (rounding error)}$$

Plot the combined element reaction forces onto a schematic diagram of the overall structure:



Problem 4 – How to code this in Python:

Given $L_1 = L_2 = 10m$, and both members have $I_1 = I_2 = 5 \times 10^{-4} m^4$, $A_1 = A_2 = 1 \times 10^{-5} m^2$ and the same elastic modulus of $E = 200 \text{ GPa}$.



The functions *local_frame* and *global_frame* are **not** inbuilt Python commands – you need to write these. Given geometric and material properties, E , A , I , and L , **for one element** the *local_frame* code generates the element stiffness matrix in local coordinates **for that one element** (refer to page 79). You can then call this function as many times as you have elements within your structure.

The function *global_frame* is then written to take the output from *local_frame* and the element orientation angle, α , generate the transformation matrix, Λ and then generate the element stiffness matrix in global coordinates, $\hat{K}$. Again, refer to page 79 for supporting equations.

Element 1	Element 2
$K1 = \text{local_frame}(E, I, A, L) =$ $K^1_{\{6 \times 6\}} = 1 \times 10^5$ $\begin{bmatrix} 2 & 0 & 0 & -2 & 0 & 0 \\ 0 & 12 & 60 & 0 & -12 & 60 \\ 0 & 60 & 400 & 0 & -60 & 200 \\ -2 & 0 & 0 & 2 & 0 & 0 \\ 0 & -12 & -60 & 0 & 12 & -60 \\ 0 & 60 & 200 & 0 & -60 & 400 \end{bmatrix}$	$K2 = \text{local_frame}(E, I, A, L) =$ $K^2_{\{6 \times 6\}} = 1 \times 10^5$ $\begin{bmatrix} 2 & 0 & 0 & -2 & 0 & 0 \\ 0 & 12 & 60 & 0 & -12 & 60 \\ 0 & 60 & 400 & 0 & -60 & 200 \\ -2 & 0 & 0 & 2 & 0 & 0 \\ 0 & -12 & -60 & 0 & 12 & -60 \\ 0 & 60 & 200 & 0 & -60 & 400 \end{bmatrix}$
$K1\text{hat}, \Lambda\text{ambda1} = \text{global_frame}(K1, \alpha1)$ $\hat{K}^1 = 1 \times 10^5$ $\begin{bmatrix} 12 & 0 & 60 & -12 & 0 & 60 \\ 0 & 2 & 0 & 0 & -2 & 0 \\ 60 & 0 & 400 & -60 & 0 & 200 \\ -12 & 0 & -60 & 12 & 0 & -60 \\ 0 & -2 & 0 & 0 & 2 & 0 \\ 60 & 0 & 200 & -60 & 0 & 400 \end{bmatrix}$	$K2\text{hat}, \Lambda\text{ambda2} = \text{global_frame}(K2, \alpha2)$ $\hat{K}^2 = 1 \times 10^5$ $\begin{bmatrix} 2 & 0 & 0 & -2 & 0 & 0 \\ 0 & 12 & 60 & 0 & -12 & 60 \\ 0 & 60 & 400 & 0 & -60 & 200 \\ -2 & 0 & 0 & 2 & 0 & 0 \\ 0 & -12 & -60 & 0 & 12 & -60 \\ 0 & 60 & 200 & 0 & -60 & 400 \end{bmatrix}$

$\Lambda^1 = \begin{bmatrix} 0 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$	$\Lambda^2 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$
Assembly Matrix (generated manually): $A^1 = \begin{matrix} & D_1' & D_2' & D_3' & D_4' & D_5' & D_6' \\ \begin{matrix} q_1 \\ q_2 \\ q_3 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$	Assembly Matrix (generated manually): $A^2 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$
$K_G^1 = A^1 \hat{K}^1 (A^1)^T =$ $K_G^1 = 1 \times 10^5 \begin{bmatrix} 12 & -60 & 0 \\ -60 & 400 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$K_G^2 = A^2 \hat{K}^2 (A^2)^T =$ $K_G^2 = 1 \times 10^5 \begin{bmatrix} 2 & 0 & 0 \\ 0 & 400 & 200 \\ 0 & 200 & 400 \end{bmatrix}$
$K_G = K_G^1 + K_G^2$ $K_G = 1 \times 10^5 \begin{bmatrix} 14 & -60 & 0 \\ -60 & 800 & 200 \\ 0 & 200 & 400 \end{bmatrix}$	
Define the global forcing vector, Q, from the information given in the question: $Q = \begin{pmatrix} 0 \\ 140,000 \\ 0 \end{pmatrix} \begin{matrix} N \\ N.m \\ N.m \end{matrix}$	
Solve for the overall structural deflections: $\begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix} = K_G^{-1} Q = \begin{pmatrix} 0.0135 \text{ m} \\ 0.0032 \text{ rad} \\ -0.0016 \text{ rad} \end{pmatrix} = \begin{pmatrix} 13.5484 \text{ mm} \\ 3.1613 \text{ mrad} \\ -1.5806 \text{ mrad} \end{pmatrix}$	
Extract the components of the structural deflections for each element, using $D^e = (A^e)^T q$ to get the element nodal deflection vectors (in global co-ordinates): $D^1 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ q_1 \\ 0 \\ q_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 13.5484 \text{ mm} \\ 0 \\ 3.1613 \text{ mrad} \end{pmatrix}$ $D^2 = \begin{pmatrix} q_1 \\ 0 \\ q_2 \\ 0 \\ 0 \\ q_3 \end{pmatrix} = \begin{pmatrix} 13.5484 \text{ mm} \\ 0 \\ 3.1613 \text{ mrad} \\ 0 \\ 0 \\ -1.5806 \text{ mrad} \end{pmatrix}$	

The element deflection vectors (in global co-ordinates) above can now be transformed into the element deflection vectors (in element co-ordinates) by the transformation matrix by: $d^e = \Lambda^e D^e$

$$d^1 = \Lambda^1 D^1 = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ q_1 \\ q_2 \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 13.5484 \text{ mm} \\ 3.1613 \text{ mrad} \end{Bmatrix} \quad d^2 = \Lambda^2 D^2 = \begin{Bmatrix} q_1 \\ 0 \\ q_2 \\ 0 \\ 0 \\ q_3 \end{Bmatrix} = \begin{Bmatrix} 13.5484 \text{ mm} \\ 0 \\ 3.1613 \text{ mrad} \\ 0 \\ 0 \\ -1.5806 \text{ mrad} \end{Bmatrix}$$

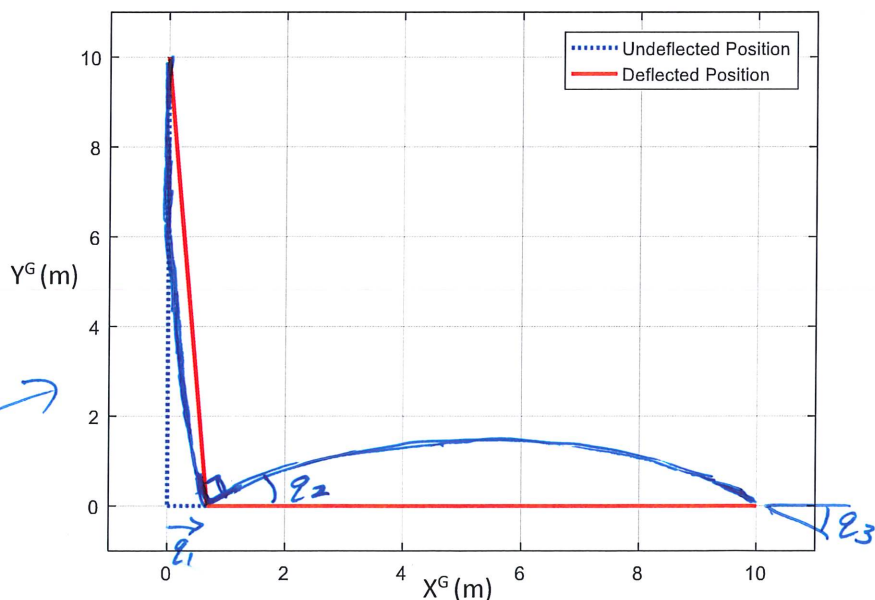
The element nodal forcing vectors (in element co-ordinates) can now be calculated by $f^e = K^e d^e$

$$f^1 = K^1 d^1 = \begin{Bmatrix} 0 \text{ kN} \\ 2.7097 \text{ kN} \\ -18.0645 \text{ kNm} \\ 0 \text{ kN} \\ -2.7097 \text{ kN} \\ 45.1613 \text{ kNm} \end{Bmatrix} \quad f^2 = K^2 d^2 = \begin{Bmatrix} 2.7097 \text{ kN} \\ 9.4839 \text{ kN} \\ 94.8387 \text{ kNm} \\ -2.7097 \text{ kN} \\ -9.4839 \text{ kN} \\ 0 \text{ kNm} \end{Bmatrix}$$

The element nodal forcing vectors (in global co-ordinates) can be calculated using either of these two methods: $F^e = \hat{K}^e D^e$ or $F^e = (\Lambda^e)^T f^e$:

$$F^1 = \begin{Bmatrix} 2.7097 \text{ kN} \\ 0 \text{ kN} \\ -18.0645 \text{ kNm} \\ -2.7097 \text{ kN} \\ 0 \text{ kN} \\ 45.1613 \text{ kNm} \end{Bmatrix} \quad F^2 = \begin{Bmatrix} 2.7097 \text{ kN} \\ 9.4839 \text{ kN} \\ 94.8387 \text{ kNm} \\ -2.7097 \text{ kN} \\ -9.4839 \text{ kN} \\ 0 \text{ kNm} \end{Bmatrix}$$

Deflected shape, using a displacement amplification of 50 (deflections 50x larger than reality):



Very Simplified and ultimately incorrect visualisation

An important note about element connections

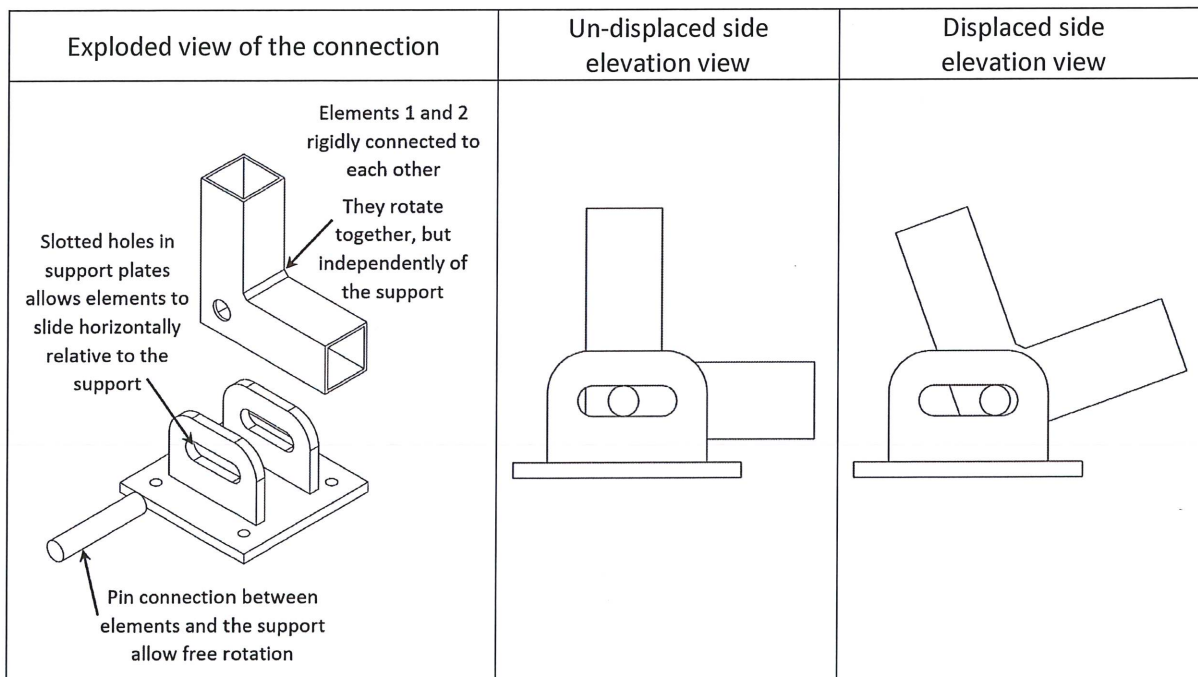
Frame elements are always assumed to be rigidly connected to one another. Any time two frame elements meet, the frame element formulation we are using assumes that these two elements are rigidly connection, such that the end of the two elements undergo the same horizontal and vertical translations, as well as the same rotation. In the example we have just done, the bottom left hand corner is shown as a roller support that does not prevent translation or rotation. We have assigned an allowable global degree of freedom at this node that corresponds to horizontal translation (we called this q_1) along with an allowable global rotation at this node (we called this q_2).

However, it is important to note that in our assembly matrices, we linked the horizontal (X^G) degree of freedom at the bottom of element 1 (D_4^1) and the horizontal (X^G) degree of freedom at the left edge of element 2 (D_1^2) to the same global degree of freedom (q_1). When we solve our system, there will be only one value within the final global deflection vector (q) that represents horizontal translation at this node, and the two elements are mathematically constrained to undergo the exact same horizontal deflection.

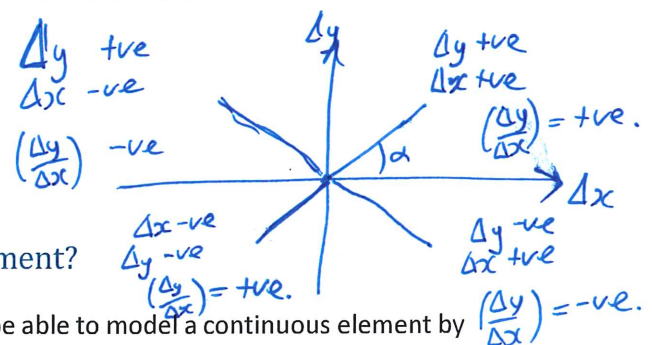
The same is true for rotations. In our assembly matrices, we linked the rotation (Z^G) degree of freedom at the bottom of element 1 (D_6^1) and the rotation (Z^G) degree of freedom at the left edge of element 2 (D_3^2) to the same global degree of freedom (q_2). Again, there will be only one value within the final global deflection vector (q) that represents rotation at this node, and the two elements are mathematically constrained to undergo the exact same rotation.

This means that the two elements translate and rotate together. There is no allowable relative translation or allowable relative rotation between the elements, even though there is an allowable translation and rotation between the elements and the supports.

Physically, this might look something like this (among several other potential options):



Do not use
numpy.arctan($\frac{\Delta y}{\Delta x}$)



How to plot the full deflected shape of a frame element?

We've said all along that the purpose of shape functions is to be able to model a continuous element by only solving for the deflections at the nodal points. Therefore, we can use these solved nodal deflections, and the shape functions that we used within our derivations, to plot the full deflected shape of a frame element. We will now go through each step required to plot this deflected shape.

Step 1: We will generate a function that plots the deflected shape of a generic frame element. We will call this function the same number of times as we have frame elements (for Problem 4, this is two).

Suggested function format, assuming that the code has been initially set up with:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
plot_deflected_shape(node1XG,node1YG,node2XG,node2YG,d_e,Disp_mag,N_points):
```

Suggested Inputs:

node1XG - the X-global (X_G) location of node 1 for the current element (relative to whatever origin location that you have chosen)
node1YG - the Y-global (Y_G) location of node 1 for the current element (relative to whatever origin location that you have chosen)
node2XG - the X-global (X_G) location of node 2 for the current element (relative to whatever origin location that you have chosen)
node2YG - the Y-global (Y_G) location of node 2 for the current element (relative to whatever origin location that you have chosen)
d_e - the solved 6×1 deflection vector for this element, in element co-ordinates
Disp_mag - a chosen displacement magnification factor to exaggerate the deflected shape
N_points - the number of points within the element at which we will evaluate the deflected shape. This number needs to be enough (minimum 10-20) if we want to obtain a smooth curve of the deflected shape.

to get L: calculate Δx and Δy and the $L = \sqrt{\Delta x^2 + \Delta y^2}$
 α can be obtained from Δx and Δy , but please use the full 4-quadrant inverse tan function.
 numpy.arctan2($\Delta y, \Delta x$)

You could also add inputs for element length, L , and element orientation angle, α , or these values can be calculated within the code from the X_G and Y_G co-ordinates of the two ends.

Step 2: We need to generate a vector of points within the element co-ordinate system (x^e). These are the points at which we will evaluate the transverse and axial deflections and eventually plot.

Suggested code: `x_e = np.linspace(0, L, N_points)`

Step 3: Using the newly defined x_e variable, evaluate the axial and shear deflection shape functions at each of these x_e points. Note that all shape functions will now be vectors, where each element of these vectors represents the shape function evaluated at "N_points" points within the elements.

Axial Shape Functions:	Transverse Shape Functions:
$\psi_1(x) = \left(1 - \frac{x}{L}\right)$	$N_1(x) = 1 - \frac{3x^2}{L^2} + \frac{2x^3}{L^3}$
$\psi_2(x) = \frac{x}{L}$	$N_2(x) = \frac{x^3}{L^2} - \frac{2x^2}{L} + x$
	$N_3(x) = \frac{3x^2}{L^2} - \frac{2x^3}{L^3}$
	$N_4(x) = \frac{x^3}{L^2} - \frac{x^2}{L}$

Step 4: Calculate the axial, $u(x)$, and transverse, $v(x)$, displacements (at all points along the element) using the shape functions, the vector of x values, and the element deflection vector, d^e (frame element numbering).

Element Axial Displacement, $u(x) = \psi_1(x)d_1 + \psi_2(x)d_4$

Element Transverse Displacement, $v(x) = N d^e = \begin{pmatrix} N_1(x) & N_2(x) & N_3(x) & N_4(x) \end{pmatrix} \begin{pmatrix} d_2 \\ d_3 \\ d_5 \\ d_6 \end{pmatrix}$

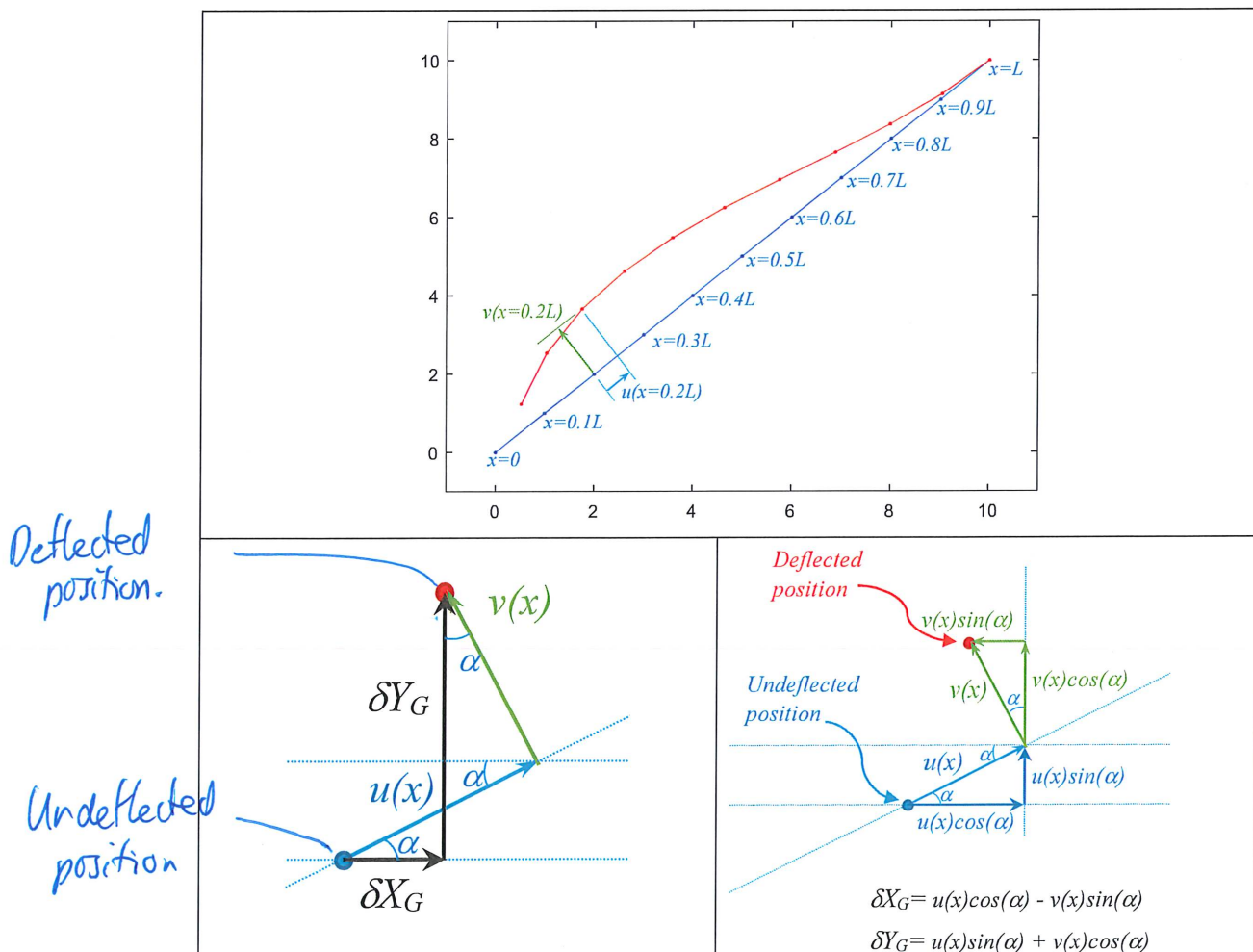
$$v(x) = N_1(x)d_2 + N_2(x)d_3 + N_3(x)d_5 + N_4(x)d_6$$

Step 5: Transform these displacements from element co-ordinates into global co-ordinates, using this subset of the transpose of the frame element transformation matrix.

$$\text{Deflections_XG} = u(x) \cos(\alpha) - v(x) \sin(\alpha)$$

$$\text{Deflections_YG} = u(x) \sin(\alpha) + v(x) \cos(\alpha)$$

These variables represent how much each of the points within the element has deflected in the X_G and Y_G directions. If you're wondering where these equations came from, consider an element inclined at angle α (in this case $\alpha \approx 45^\circ$) and the internal deflection evaluated at 11 points ($N_points = 11$), which will break the element up into 10 segments. The $u(x)$ and $v(x)$ values evaluated in Step 4 represent the axial and transverse deflections of each internal point within the element, in element co-ordinates.



Step 6: Generate a baseline vector of points in X_G and Y_G that represent the un-deflected baseline of the element.

Suggested code:

```
Undeformed_baseline_XG = np.linspace(node1XG,node2XG,N_points)
Undeformed_baseline_YG = np.linspace(node1YG,node2YG,N_points)
```

Step 7: Calculate a vector of points in X_G and Y_G that represent the deflected position of all points within the element.

Suggested code:

```
Deflected_XG = Undeformed_baseline_XG + Disp_mag*Deflections_XG
Deflected_YG = Undeformed_baseline_YG + Disp_mag*Deflections_YG
```

Step 8: Plot the result (using your define prefix, such as `plt.plot(...)`).

```
plt.plot(Undeformed_baseline_XG,Undeformed_baseline_YG,'b.-')
plt.plot(Deflected_XG,Deflected_YG,'r.-')
```

Using this approach the plot of the deflected position of the structure in Problem 4 now looks like (using a displacement multiplier of 100):

